

pca

May 28, 2025

```
[1]: from sklearn.datasets import fetch_olivetti_faces
```

```
X, y = fetch_olivetti_faces(return_X_y=True)
```

```
[2]: import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
```

```
print(f'Original shape: {X.shape}')
```

```
pca = PCA(n_components=20)
X_pca = pca.fit_transform(X)
```

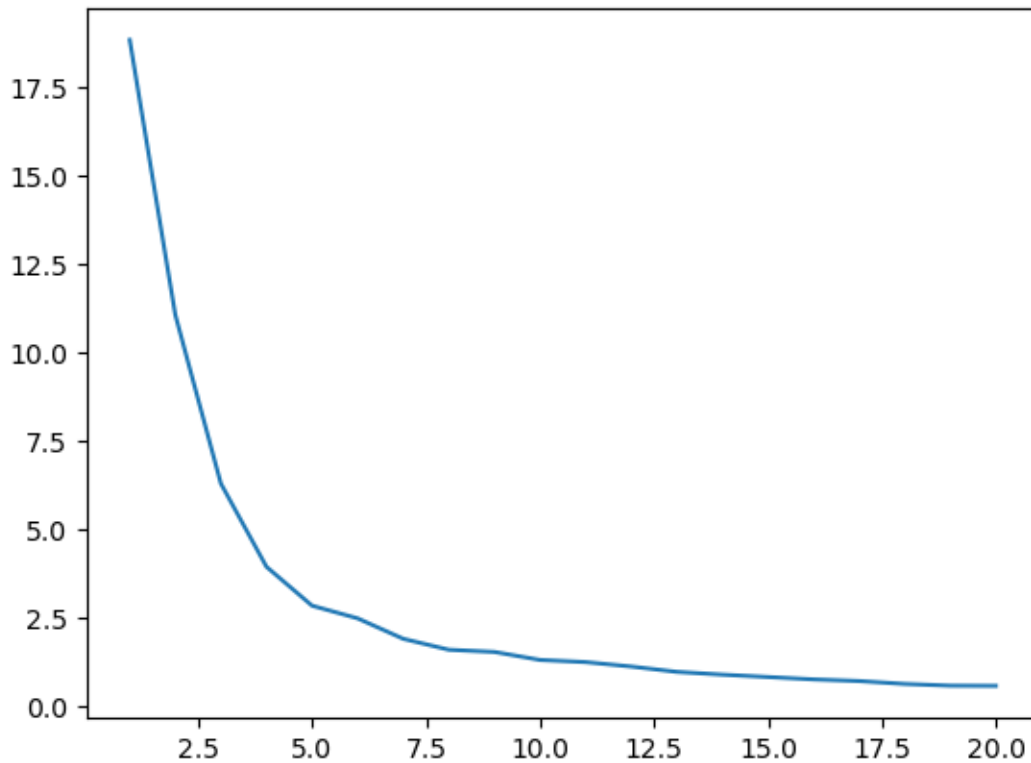
```
print(f'PCA shape: {X_pca.shape}')
```

```
eigenvalues = pca.explained_variance_
```

```
plt.figure()
plt.plot(np.arange(1, len(eigenvalues) + 1), np.sort(eigenvalues)[::-1])
plt.show()
```

Original shape: (400, 4096)

PCA shape: (400, 20)



```
[3]: pca = PCA(n_components=5)
```

```
X_pca = pca.fit_transform(X)
```

```
[4]: from sklearn.model_selection import cross_val_score, StratifiedKFold  
from sklearn.naive_bayes import GaussianNB
```

```
scores = cross_val_score(GaussianNB(), X_pca, y,  
    ↪cv=StratifiedKFold(n_splits=10, shuffle=True))
```

```
print(f'10-fold cross-validated accuracy: {scores.mean():.4f} ± {scores.std():.  
    ↪4f}')  
10-fold cross-validated accuracy: 0.6050 ± 0.0472
```